

Zero to Hero Study Handbook: pytorch-fraud-mlp

This handbook is built from static analysis of this repository only.

Module 1: Foundations & Architecture

1.1 What this project does

This project builds and evaluates a fraud detection model for highly imbalanced tabular data (credit card transactions). It centers on a PyTorch MLP and compares it against scikit-learn baselines.

Main use cases in this repo: - Train and evaluate a fraud classifier in a generated notebook (`pytorch_fraud_mlp.ipynb`, produced by `generate_notebook.py`). - Compare imbalance strategies, especially threshold tuning vs weighted loss. - Run batch inference on new CSV data using `inference.py` and export predictions.

1.2 Core paradigms and patterns used here

Definitions first: - Object-oriented programming (OOP): behavior and state are grouped in classes. - Functional decomposition: reusable functions each do one step of a pipeline. - Pipeline-oriented workflow: data moves through ordered preprocessing, training, evaluation, and reporting stages. - CLI-oriented execution: command-line arguments control input/output and runtime behavior. - Configuration by dataclass: hyperparameters and paths are centralized in one typed structure.

How these appear in this codebase: - OOP: `FraudMLP` class in both `generate_notebook.py` and `inference.py`; `FraudDataset` class in notebook code authored inside `generate_notebook.py`. - Functional style: `run_epoch`, `predict`, `tune_threshold`, `train_variant`, `preprocess`, `run_inference`, `load_model`. - Pipeline flow: data load -> feature scaling -> train/val/test split -> `DataLoader` -> model training -> threshold tuning -> metrics/plots. - CLI pattern: `inference.py` uses `argparse` with `--input`, `--output`, `--checkpoint`, `--threshold`, `--device`. - Config pattern: `Config` dataclass defines paths and hyperparameters in notebook content built by `generate_notebook.py`.

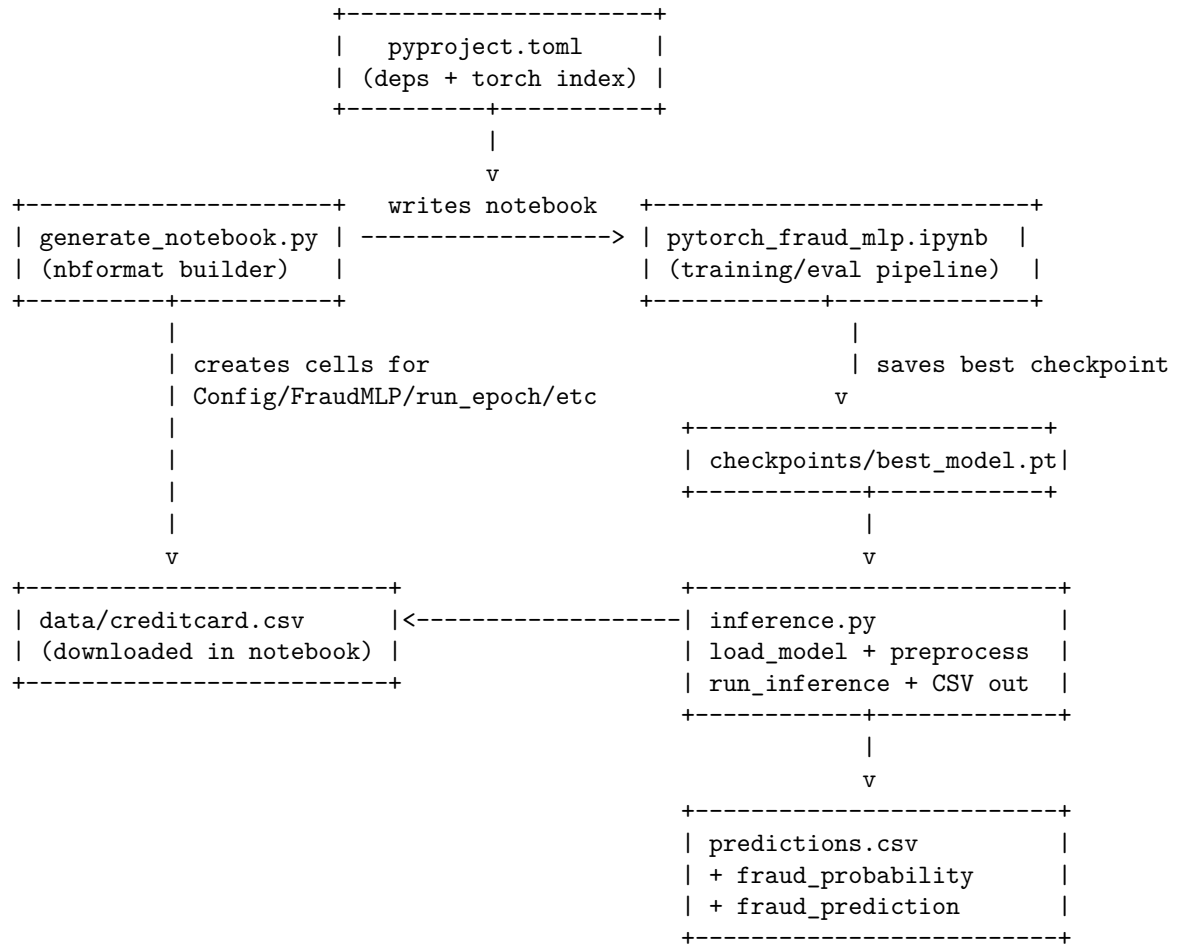
1.3 Architecture overview

Primary components: - Notebook generator (`generate_notebook.py`): creates the full training/evaluation notebook programmatically via `nbformat`. - Notebook runtime (`pytorch_fraud_mlp.ipynb`): contains the end-to-end ML workflow. - Inference CLI (`inference.py`): loads saved checkpoint and predicts on new CSV data. - Project configuration (`pyproject.toml`): dependency and package index setup for `uv`.

Main architecture interactions: 1. `generate_notebook.py` writes `pytorch_fraud_mlp.ipynb`. 2. Notebook (when run by user) downloads data from Kaggle if needed,

trains model, and saves `checkpoints/best_model.pt`. 3. `inference.py` loads `checkpoints/best_model.pt`, preprocesses incoming CSV, and writes predictions CSV.

ASCII architecture flow:



Module 2: Repository Map

Focus files to learn first are listed first.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>generate_notebook.py</code>	Programmatically builds the full training/evaluation notebook with markdown and code cells	<code>md</code> , <code>code</code> (file-level helpers), notebook cell code contains <code>Config</code> , <code>FraudDataset</code> , <code>FraudMLP</code> , <code>run_epoch</code> , <code>predict</code> , <code>tune_threshold</code> , <code>train_variant</code>	<code>SEED=42</code> , <code>cfg.hidden_dims=[256,128,64]</code> , <code>cfg.dropout=0.3</code> , <code>cfg.batch_size=512</code> , <code>cfg.lr=1e-3</code> , <code>cfg.weight_decay=1e-4</code> , <code>cfg.epochs=60</code> , <code>cfg.patience=10</code>
<code>inference.py</code>	CLI inference pipeline on new transaction CSVs	<code>FraudMLP</code> , <code>load_model</code> , <code>preprocess</code> , <code>run_inference</code> , <code>main</code>	<code>FEATURE_COLS = V1..V28 + Amount_s + Time_s</code> ; CLI args <code>--input</code> , <code>--output</code> , <code>--checkpoint</code> , <code>--threshold</code> , <code>--device</code>
<code>pytorch_fraud_mlp.py</code>	Main notebook artifact for end-to-end training, evaluation, and visualization	Contains generated cell code from <code>generate_notebook.py</code>	Uses checkpoint path <code>checkpoints/best_model.pt</code> and figure outputs under <code>figures/</code>
<code>README.md</code>	Human-readable project overview, setup commands, and run instructions	N/A	Documents Kaggle credential options (<code>~/kaggle/access_token</code> or <code>KAGGLE_TOKEN</code>) and uv commands

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>pyproject.toml</code>	Project metadata and dependencies for <code>uv</code>	N/A	<code>requires-python >=3.12.10</code> ; dependencies (torch, numpy, pandas, scikit-learn, matplotlib, seaborn, jupyter, ipykernel, nbformat, tqdm, kaggle); <code>tool.uv.sources.torch =</code> <code>pytorch-cu128</code> Lockfile used by <code>uv sync</code>
<code>uv.lock</code>	Locked dependency versions for reproducible installs	N/A	
<code>figures/</code>	Stores generated plots from notebook	N/A	Expected outputs include <code>eda_overview.png</code> , <code>training_curves.png</code> , <code>pr_and_threshold.png</code> , <code>confusion_matrix.png</code> , <code>model_comparison.png</code> , <code>pr_curve_comparison.png</code> <code>creditcard.csv</code> expected path: <code>data/creditcard.csv</code>
<code>data/</code> (runtime-created)	Dataset storage location created by <code>Config.__post_init__</code> in notebook code	N/A	
<code>checkpoints/</code> (runtime-created)	Model checkpoint storage created by <code>Config.__post_init__</code> in notebook code	N/A	Best model checkpoint: <code>checkpoints/best_model.pt</code>

Module 3: Core Execution Flows

Flow A: Notebook generation flow (`generate_notebook.py`)

Purpose: - Build `pytorch_fraud_mlp.ipynb` from code-defined cell templates.

Step-by-step: 1. Defines helper wrappers `md(src)` and `code(src)` for notebook cell creation. 2. Populates a large `cells` list containing markdown and code strings. 3. Creates notebook object using `nbf.v4.new_notebook()`. 4. Injects kernel/language metadata (`display_name: pytorch-fraud-mlp`, `python 3.12.10`). 5. Writes notebook to `pytorch_fraud_mlp.ipynb`.

Input/Output shapes for this flow: - Input: no external runtime arguments. - Output artifact: one notebook file at path `pytorch_fraud_mlp.ipynb`.

Flow B: Training and evaluation flow (inside generated notebook)

Purpose: - Download/load data, train MLP, evaluate with PR-AUC-centric workflow, compare baselines.

Step-by-step with real function/class names: 1. Repro setup and device selection. - Sets `SEED = 42`, NumPy and PyTorch seeds, deterministic cuDNN flags. - Chooses `device = cuda` if available else `cpu`. 2. Configuration. - `Config` dataclass centralizes paths and hyperparameters. - `__post_init__` ensures `data/`, `checkpoints/`, `figures/` exist. 3. Data loading. - If `data/creditcard.csv` missing, calls Kaggle CLI download command through `subprocess.run`. - Reads CSV via `pd.read_csv`. 4. Preprocessing and split. - Creates `Amount_s` and `Time_s` with `StandardScaler`. - Builds `FEATURE_COLS = [V1..V28, Amount_s, Time_s]`. - Creates `X (float32)` and `y (float32)`. - Performs stratified train/val/test split via `train_test_split`. 5. Dataset and dataloaders. - `FraudDataset` stores tensors. - `train_loader`, `val_loader`, `test_loader` are constructed. 6. Model and optimizer stack. - `FraudMLP` with hidden dims `[256, 128, 64]`, `BatchNorm1d`, `GELU`, `Dropout(0.3)`, final 1-logit output. - `criterion = nn.BCEWithLogitsLoss()`. - `optimizer = AdamW(...)` and `scheduler = CosineAnnealingLR(...)`. 7. Training loop. - `run_epoch` handles both train and eval mode (`optimizer` is `None` for eval). - Tracks history values (`train_loss`, `val_loss`, `train_ap`, `val_ap`, `lr`). - Saves best checkpoint on improved validation PR-AUC. 8. Evaluation and threshold tuning. - Loads best checkpoint. - `predict` computes probabilities on val/test sets. - `tune_threshold` chooses threshold that maximizes F1 on validation PR curve. - Applies tuned threshold to test predictions. 9. Ablation and baselines. - `train_variant` evaluates different `pos_weight` settings. - Trains Logistic Regression and Random Forest baselines. - Builds `results` DataFrame for metric comparison.

Key data structures and exact key sets:

1. Feature matrix and labels:

```
FEATURE_COLS = [f"V{i}" for i in range(1, 29)] + ["Amount_s", "Time_s"]
X = df[FEATURE_COLS].to_numpy(dtype=np.float32) # shape: (N, 30)
y = df["Class"].to_numpy(dtype=np.float32)      # shape: (N,)
```

2. `FraudDataset` sample return contract:

- `__getitem__(idx)` returns `(X_i, y_i)` where:
 - `X_i`: `torch.Tensor` with 30 features.
 - `y_i`: `torch.Tensor` with shape `(1,)` due to `unsqueeze(1)` at dataset construction.
3. Checkpoint dictionary written by training loop (`torch.save`):
- `epoch` (int)
 - `model_state_dict` (state dict)
 - `optimizer_state_dict` (state dict)
 - `val_pr_auc` (float)
 - `hidden_dims` (list[int])
 - `dropout` (float)
 - `input_dim` (int)
 - `feature_cols` (list[str])
4. Core function return contracts in notebook code:
- `run_epoch(...)` -> `Tuple[float, float]` returns `(avg_loss, pr_auc)`.
 - `predict(...)` -> `Tuple[np.ndarray, np.ndarray]` returns (probabilities, labels).
 - `tune_threshold(...)` -> `Tuple[float, float]` returns `(best_threshold, best_f1)`.
 - `train_variant(...)` -> dict returns keys: `pos_weight`, `val_PR_AUC`, `test_PR_AUC`, `threshold`, `test_F1`.

Flow C: Inference CLI flow (`inference.py`)

Purpose: - Load saved model and run fraud predictions on an external CSV.

Step-by-step: 1. Parse CLI args in `main()`. 2. Resolve runtime device (`auto`, `cuda`, or `cpu`). 3. Validate checkpoint path and load with `load_model`. 4. Read input CSV with `pd.read_csv`. 5. Preprocess in `preprocess(df)`: - Creates `Amount_s` and `Time_s` from input-file statistics. - Validates required columns are present. - Returns `np.float32` feature matrix using `FEATURE_COLS`. 6. Run batched scoring via `run_inference`: - Applies model, sigmoid, and thresholding. 7. Write output CSV: - Copies original rows. - Adds `fraud_probability` and `fraud_prediction`.

Exact input/output schema:

Input CSV contract: - Required columns: `V1 ... V28`, `Amount`, `Time`. - Optional extra columns: allowed (example: `Class` may exist and is ignored by inference logic).

Internal inference tensor flow: - `preprocess` output shape: `(N, 30)` float32 array. - `run_inference` output: - `probs`: shape `(N,)`, float probabilities in `[0,1]`. - `preds`: shape `(N,)`, integer 0/1 via `(probs >= threshold).astype(int)`.

Output CSV columns: - All original input columns. - `fraud_probability` (rounded to 6 decimals before write). - `fraud_prediction` (0 or 1).

Module 4: Setup & Run Guide

This section documents how to use the repository on a clean machine, based on `README.md` and project manifests.

4.1 Prerequisites

- Linux environment.
- `uv` installed.
- Python compatible with `>=3.12.10` (from `pyproject.toml`).
- Kaggle CLI auth for auto dataset download.

4.2 Dependency installation

```
git clone https://github.com/pypi-ahmad/pytorch-fraud-mlp.git
cd pytorch-fraud-mlp
uv sync --python 3.12.10
```

Dependency source details: - PyTorch is pulled from `https://download.pytorch.org/whl/cu128` via `tool.uv.index` and `tool.uv.sources`.

4.3 Environment and credentials

Required credential path or env key for data download flow: - `~/.kaggle/access_token` (README-documented file-based token), or - `KAGGLE_TOKEN` environment variable.

Example from README:

```
mkdir -p ~/.kaggle
echo 'YOUR_KAGGLE_TOKEN' > ~/.kaggle/access_token
chmod 600 ~/.kaggle/access_token
# OR
export KAGGLE_TOKEN=YOUR_KAGGLE_TOKEN
```

4.4 Typical command sequence

1. Generate notebook file:

```
uv run python generate_notebook.py
```

2. Register kernel:

```
uv run python -m ipykernel install --user --name pytorch-fraud-mlp --display-name "pytorch-f
```

3. Open notebook:

```
uv run jupyter notebook pytorch_fraud_mlp.ipynb
```

```
# or
```

```
uv run jupyter lab pytorch_fraud_mlp.ipynb
```

4. Run inference (after training has produced checkpoint):

```
uv run python inference.py --input new_transactions.csv --output predictions.csv --threshold
```

4.5 Migration/seeding/external services

- Database migrations: none found in this repository.
- Seeding scripts: none found.
- External service integration: Kaggle dataset download is invoked in notebook code through Kaggle CLI subprocess call.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

1. Read `README.md` to understand problem framing, metrics, and high-level workflow.
2. Read `pyproject.toml` to understand runtime dependencies and package index configuration.
3. Read `inference.py` end-to-end first. It is short and shows the minimal production-style path from model load to CSV output.
4. Read `generate_notebook.py` sections in this order:
 - `Config` + data loading code
 - preprocessing + `FraudDataset`
 - `FraudMLP` + training setup
 - `run_epoch/predict` + training loop
 - `tune_threshold` + ablation + baseline comparison
5. Open `pytorch_fraud_mlp.ipynb` to see how the generated flow is presented interactively.
6. Reconcile notebook narrative claims with the exact code paths in `generate_notebook.py`.

5.2 Practice exercises (with solution outlines)

Exercise 1: - Task: List all checkpoint keys expected by `inference.py` and identify which are mandatory for successful load. - Where to inspect: `generate_notebook.py` training checkpoint save block and `inference.py::load_model`. - Solution outline: `load_model` requires at minimum `hidden_dims`, `dropout`, and `model_state_dict`; `input_dim` has fallback to `len(FEATURE_COLS)`. Training also saves `epoch`, `optimizer_state_dict`, `val_pr_auc`, and `feature_cols`.

Exercise 2: - Task: Explain why `inference.py` requires `Amount` and `Time` even though model features are `Amount_s` and `Time_s`. - Where to in-

spect: `inference.py::preprocess` and `FEATURE_COLS`. - Solution outline: `preprocess` computes standardized `Amount_s` and `Time_s` from raw `Amount` and `Time`; it then selects `FEATURE_COLS` including standardized fields.

Exercise 3: - Task: Trace the no-leakage threshold workflow in notebook code. - Where to inspect: `predict`, `tune_threshold`, and threshold section in generated notebook cell code. - Solution outline: threshold is fit on validation predictions (`val_probs`, `val_labels`) and then applied unchanged on test predictions (`test_probs`), preventing test-label leakage.

Exercise 4: - Task: Write the exact model architecture in order, including normalization, activation, and dropout. - Where to inspect: `FraudMLP` in both `generate_notebook.py` and `inference.py`. - Solution outline: repeated block `Linear` -> `BatchNorm1d` -> `GELU` -> `Dropout(0.3)` for hidden layers [256, 128, 64], followed by final `Linear(..., 1)` logit output.

Exercise 5: - Task: Identify every place where class imbalance is handled or evaluated. - Where to inspect: `DataLoader` section, `BCEWithLogitsLoss`, `tune_threshold`, and `train_variant`. - Solution outline: main path uses plain BCE + threshold tuning; imbalance weighting is computed (`pos_weight`) and tested in ablation variants rather than used by default.

Exercise 6: - Task: Describe `run_epoch` behavior differences between train and eval mode. - Where to inspect: `run_epoch` function. - Solution outline: train mode (`optimizer` present) does backward pass, gradient clipping, and optimizer step; eval mode (`optimizer=None`) runs under `torch.no_grad()` and skips updates.

Exercise 7: - Task: Enumerate all CLI options for inference and their defaults. - Where to inspect: `inference.py::main` argparse setup. - Solution outline: `--input` required; `--output` default `predictions.csv`; `--checkpoint` default `checkpoints/best_model.pt`; `--threshold` default 0.5; `--device` choices `auto|cuda|cpu` default `auto`.

Exercise 8: - Task: Explain why this repo uses PR-AUC as primary selection metric and where this is enforced in code. - Where to inspect: README metric discussion and `run_epoch`/training loop using `average_precision_score` and best checkpoint by `val_pr_auc`. - Solution outline: PR-AUC is computed in `run_epoch`; training loop chooses best model by validation PR-AUC; README and notebook narrative explain ROC-AUC inflation under extreme imbalance.

Learner Verification Checklist

Use this self-check after completing the study plan:

- Can you explain, without looking, the three main artifacts and their roles: `generate_notebook.py`, `pytorch_fraud_mlp.ipynb`, and `inference.py`?

- Can you reproduce the exact **FraudMLP** layer sequence and its hyperparameters?
- Can you describe why validation-based threshold tuning avoids leakage and where it occurs in code?
- Can you list the checkpoint keys and explain how **inference.py** consumes them?
- Can you state the exact input and output CSV schema for inference?
- Can you justify why PR-AUC is primary in this repository and point to the functions that compute and use it?
- Can you identify one production caveat in **inference.py** preprocessing and explain how to fix it (training-scaler serialization)?
- Can you describe how baselines (Logistic Regression, Random Forest) are integrated into the same evaluation framework?